

Part A, Topic 2 - Olympic Tweets



MONASH University

Lance Belen

June 2, 2025

Contents

1	Data Wrangling, Visualisation, and Analysis	3
1.1	Data Wrangling	4
1.2	Visualisations and Analysis	7
2	Predictive Data Analysis	16
2.1	Model Training	16
2.2	Model Evalutaion	24
2.3	Final Model Improvements	25
2.4	Predictions on the Testing Set	27
	References	29

1 Data Wrangling, Visualisation, and Analysis

First, load the packages needed.

```
options(repos = c(CRAN = "https://cloud.r-project.org/"))

if (!requireNamespace("tidyverse", quietly = TRUE)) {
  install.packages("tidyverse")
}
if (!requireNamespace("lubridate", quietly = TRUE)) {
  install.packages("lubridate")
}
if (!requireNamespace("impute", quietly = TRUE)) {
  install.packages("impute")
}
if (!requireNamespace("dplyr", quietly = TRUE)) {
  install.packages("dplyr")
}
if (!requireNamespace("magrittr", quietly = TRUE)) {
  install.packages("magrittr")
}
if (!requireNamespace("ggplot2", quietly = TRUE)) {
  install.packages("ggplot2")
}
if (!requireNamespace("readxl", quietly = TRUE)) {
  install.packages("readxl")
}
if (!requireNamespace("stringr", quietly = TRUE)) {
  install.packages("stringr")
}
if (!requireNamespace("forcats", quietly = TRUE)) {
  install.packages("forcats")
}
if (!requireNamespace("tidytext", quietly = TRUE)) {
  install.packages("tidytext")
}
if (!requireNamespace("xgboost", quietly = TRUE)) {
  install.packages("xgboost")
}
if (!requireNamespace("MLmetrics", quietly = TRUE)) {
  install.packages("MLmetrics")
}

library(tidyverse)
library(lubridate)
library(impute)
library(dplyr)
library(magrittr)
library(ggplot2)
library(readxl)
library(stringr)
library(forcats)
library(tidytext)
library(xgboost)
```

```
library(MLmetrics)
```

1.1 Data Wrangling

Then, read and open the dataset.

```
tweets <- read.csv("Olympics_tweets.csv")
```

Next, explore the dataset.

```
cat("Number of rows: ", nrow(tweets), "\n")
cat("Number of columns: ", ncol(tweets), "\n")
knitr::kable(head(tweets))
cat("Structure of the dataset: \n")
str(tweets)
cat("Summary of the dataset: \n")
summary(tweets)
```

```
## Number of rows: 114213
```

```
## Number of columns: 13
```

id	text	user	screen_name	location	verified	description	user_created_at	tweet_created_at	language
1.418962e+18	Mirabai Chanu's maiden Olympic silver helped India clinch joint-12th spot on the medals tally on Day 1 atâ€¦ https://t.co/ksy8OvaJrl	ITGD_News	ITGD_News	India	FALSE	Live cricket scores- news-analysis and fun facts on all your favourite sports.	26/09/2020 10:57	24/07/2021 15:52	en
1.418962e+18	Mirabai Chanu brings home first silver medal on Day 1. DSEU congratulates her on this achievement. A brilliant stâ€¦ https://t.co/allvBeujfH	dseu_official	dseu_official		FALSE	The official Twitter account for Delhi Skill and Entrepreneurship Univeristy.	12/11/2020 9:01	24/07/2021 15:52	en
1.418962e+18	congratulations to Mirabai Chanu for starting the medal tally for India in the Tokyo Olympics 2020 by wiâ€¦ https://t.co/NukYganXeA	SutejNarwar	SutejNarwar	Delhi-India	FALSE	Believe in yourself@ðŸ™ŒðŸ™ŒðŸ™Œ	20/04/2020 16:56	24/07/2021 15:52	en

id	text	user_screen_name	user_location	retweet_count	favorite_count	user_description	user_created_at	user_followers	user_friends	date	language
1.418962e+18	Happy 18th Congratulations to ace Indian weightlifter (mirabai_chanu?) for winning a Silver in the womenâ€™s #Weightliftingâ€¦ https://t.co/QJqHGstelZ	NsChandapaka	Nalgonda- Telangana	0	1	FALSE Govt Teacher YouTuber Ambedkarist Indian Cricket Lover Founder of (BORN4WIN?) Social Activist Kargil Vijay Diwas	8/08/2017 1:26	760	24	07/2021 15:53	en
1.418963e+18	Happy 18th Congratulations to (mirabai_chanu?) for winning silver medal in weight lifting. This is first medal for India in Tokyoâ€¦ https://t.co/NhVphfI01A	Prabhatpur	Uttar Pradesh	0	1	FALSE Nationalist Engineer Plastics Technologist 'CIPET'ian	14/11/2016 17:13	1653	24	07/2021 15:53	en
1.418964e+18	Happy 18th (Narendra modi?) (Tokyo2020?) (mirabai_chanu?) Noble Group congratulates India's Pride Mirabai Chanu on her winning ist Tâ€™s https://t.co/4BizUPL60n	JaiHaryana	India	0	1	FALSE MBA(MDU)- FORMER GM HARYANA	29/07/2018 16:19	184	24	07/2021 15:53	en

Structure of the dataset:

```
## 'data.frame':   114213 obs. of  13 variables:
## $ id          : num  1.42e+18 1.42e+18 1.42e+18 1.42e+18 1.42e+18 ...
## $ text        : chr  "Mirabai Chanu's maiden Olympic silver helped India clinch joint-12th spot
## $ user_screen_name: chr  "ITGDsports" "dseu_official" "SutejKumar4" "NsChandapaka" ...
## $ user_location  : chr  "Noida- India" NA "New Delhi- India" "Nalgonda- Telangana" ...
## $ retweet_count  : int  1 0 0 1 0 0 0 1 0 0 ...
## $ favorited      : logi  FALSE FALSE FALSE FALSE FALSE FALSE ...
## $ favorite_count  : int  1 1 0 0 1 0 0 2 0 0 ...
## $ user_description: chr  "Live cricket scores- news- analysis and fun facts on all your favourite sports
## $ user_created_at : chr  "26/09/2017 10:57" "12/11/2020 9:01" "20/04/2020 16:56" "8/08/2017 1:26" ...
## $ user_followers  : int  9333 1168 72 5907 2721 1 61 7247 180 771 ...
## $ user_friends    : int  28 4 143 60 353 84 405 185 9 1071 ...
## $ date           : chr  "24/07/2021 15:52" "24/07/2021 15:52" "24/07/2021 15:52" "24/07/2021 15:53
## $ language       : chr  "en" "en" "en" "en" ...
```

Summary of the dataset:

```
##           id           text           user_screen_name  user_location
## Min.      :1.419e+18  Length:114213      Length:114213      Length:114213
## 1st Qu.:1.420e+18  Class :character  Class :character  Class :character
## Median :1.420e+18  Mode  :character  Mode  :character  Mode  :character
## Mean      :1.420e+18
## 3rd Qu.:1.421e+18
```

```
## Max.      :1.422e+18
##
## retweet_count      favorited      favorite_count      user_description
## Min.       : 0.0000      Mode :logical      Min.       : 0.00      Length:114213
## 1st Qu.: 0.0000      FALSE:114210      1st Qu.: 0.00      Class :character
## Median : 0.0000      NA's :3          Median : 0.00      Mode  :character
## Mean      : 0.3502                      Mean      : 2.13
## 3rd Qu.: 0.0000                      3rd Qu.: 1.00
## Max.      :2404.0000                  Max.      :9572.00
## NA's      :3                          NA's      :3
## user_created_at      user_followers      user_friends      date
## Length:114213      Min.       : 0      Min.       : 0      Length:114213
## Class :character      1st Qu.: 106      1st Qu.: 179      Class :character
## Mode  :character      Median : 456      Median : 498      Mode  :character
##                      Mean      : 92012      Mean      : 1429
##                      3rd Qu.: 2007      3rd Qu.: 1188
##                      Max.      :78061816      Max.      :772902
##                      NA's      :9          NA's      :9
## language
## Length:114213
## Class :character
## Mode  :character
##
##
##
##
```

Now I will check the dataset for missing or incorrect values that needs handling.

```
# retweet_count
tweets <- tweets %>%
  mutate(retweet_count = ifelse(retweet_count < 0, NA, retweet_count)) %>%
  mutate(retweet_count = replace_na(retweet_count, 0))

# favorite_count
tweets <- tweets %>%
  mutate(favorite_count = ifelse(favorite_count < 0, NA, favorite_count)) %>%
  mutate(favorite_count = replace_na(favorite_count, 0))

# user_followers
tweets <- tweets %>%
  mutate(user_followers = ifelse(user_followers < 0, NA, user_followers)) %>%
  mutate(user_followers = replace_na(user_followers, 0))

# user_friends
tweets <- tweets %>%
  mutate(user_friends = ifelse(user_friends < 0, NA, user_friends)) %>%
  mutate(user_friends = replace_na(user_friends, 0))

# user_created_at
tweets <- tweets %>%
  mutate(user_created_at = dmy_hm(user_created_at)) %>%
  filter(
    user_created_at >= as.POSIXct("2006-03-21"),
```

```

    user_created_at <= Sys.time()
  )

# date
tweets <- tweets %>%
  mutate(date = dmy_hm(date)) %>%
  filter(
    date >= as.POSIXct("2006-03-21"),
    date <= Sys.time()
  )

```

1.2 Visualisations and Analysis

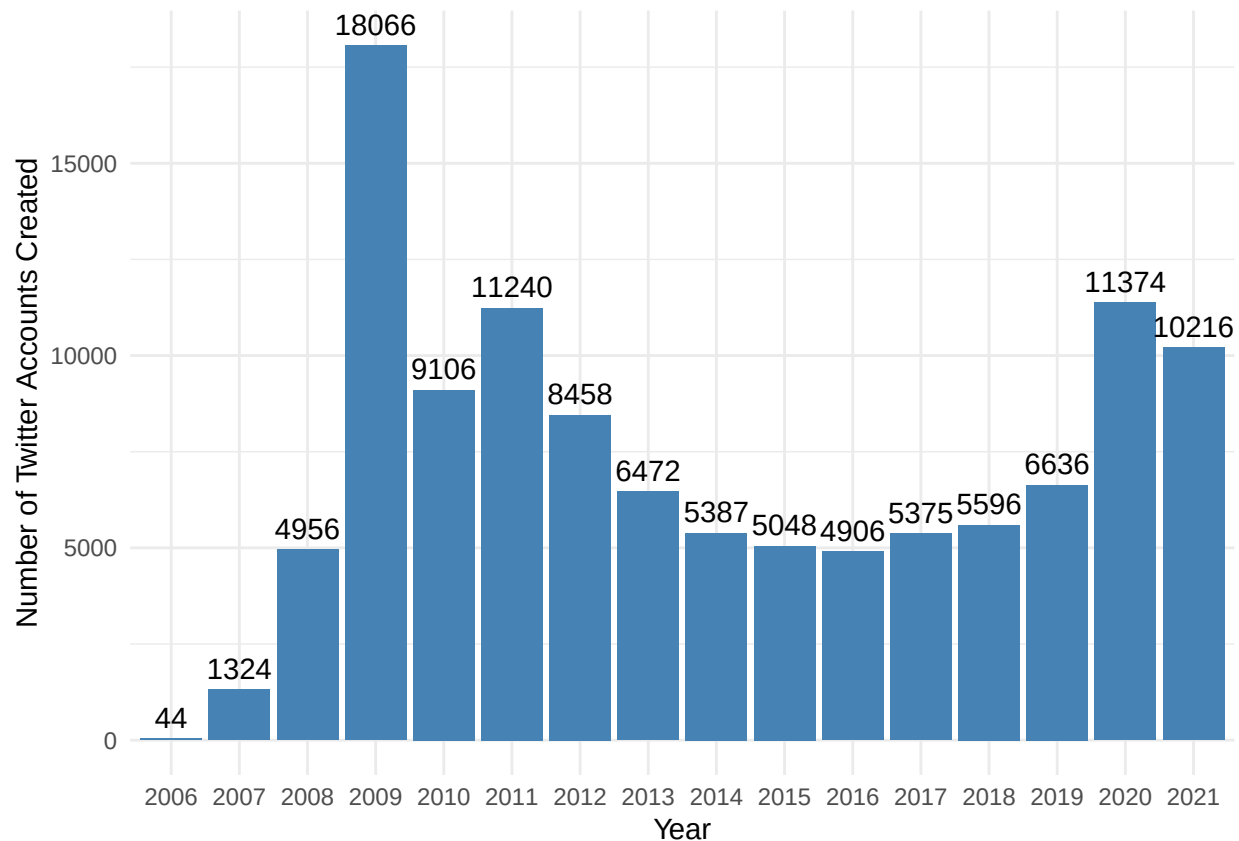
1.2.1 Number of Twitter Accounts Created per Year

```

# Create a new column for the year
tweets$year <- year(tweets$user_created_at)
tweets$year <- as.integer(tweets$year)
tweets$year <- as.factor(tweets$year)

# Bar plot of the number of Twitter accounts per year
ggplot(tweets, aes(x = year)) +
  geom_bar(fill = "steelblue") +
  geom_text(
    stat = "count", aes(label = ..count..),
    position = position_dodge(width = 0.9),
    vjust = -0.5) +
  labs(x = "Year", y = "Number of Twitter Accounts Created") +
  theme_minimal()

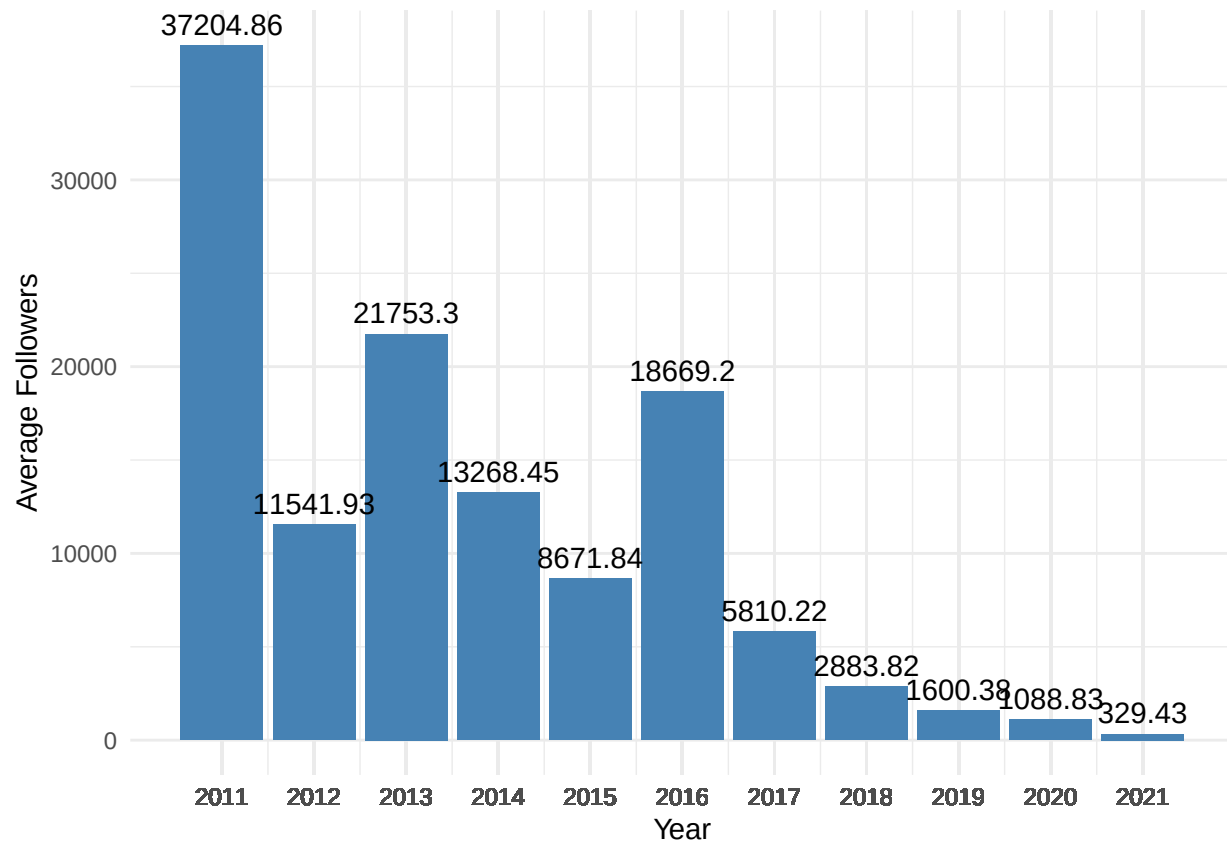
```



1.2.2 Average Followers per Year

```
creators_after_2010 <- tweets
creators_after_2010 <- creators_after_2010 %>% mutate(year = as.numeric(as.character(year)))
creators_after_2010 <- creators_after_2010 %>% filter(year > 2010)

creators_after_2010 %>%
  filter(year > 2010) %>%
  group_by(year) %>%
  summarise(avg_followers = mean(user_followers, na.rm = TRUE)) %>%
  ggplot(aes(x = year, y = avg_followers)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  geom_text(
    aes(label = round(avg_followers, 2)),
    vjust = -0.5) +
  scale_x_continuous(breaks = creators_after_2010$year) +
  labs(x = "Year", y = "Average Followers") +
  theme_minimal()
```

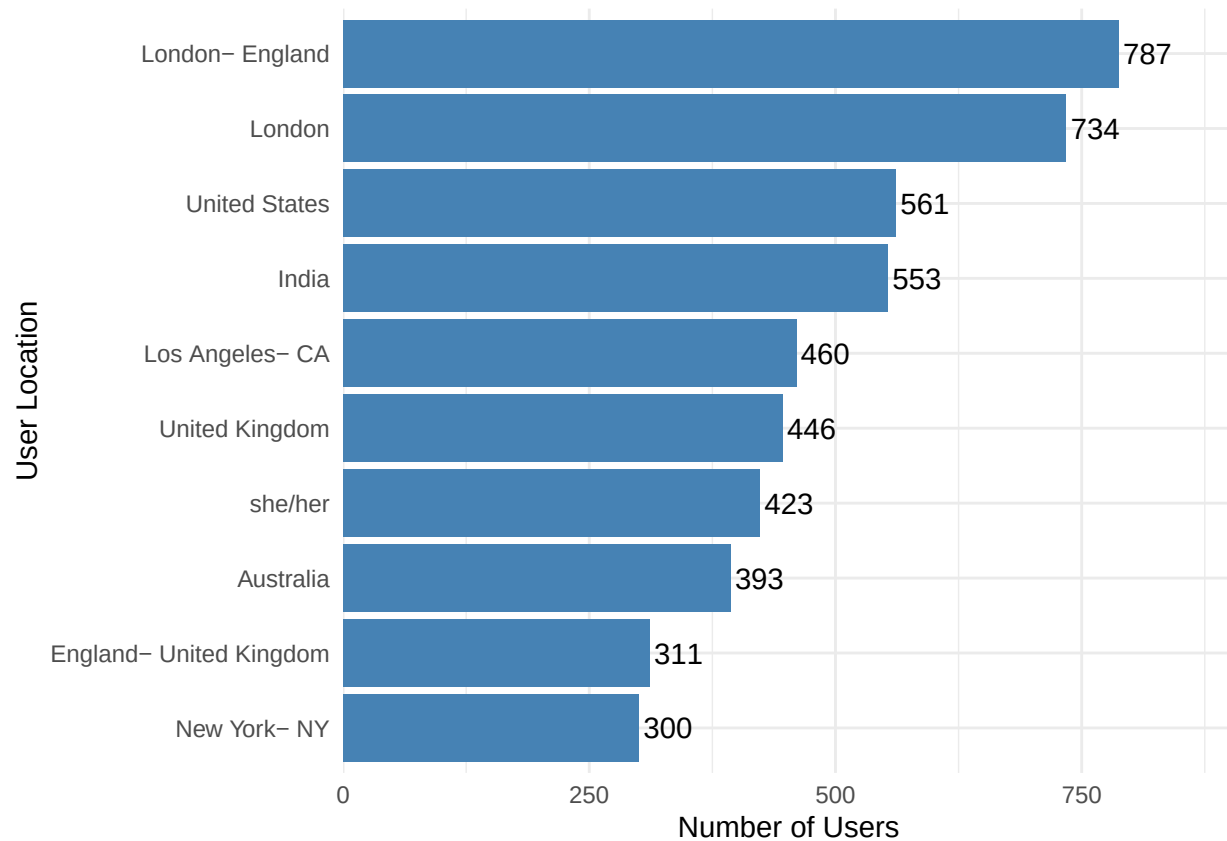



1.2.3 Top 10 Most Frequent Location of Users

```
top_locations <- tweets %>%
  select(user_screen_name, user_location) %>%
  distinct() %>%
  filter(!is.na(user_location), user_location != "") %>%
  count(user_location, name="user_count") %>%
  arrange(desc(user_count)) %>%
  dplyr::slice(1:10)

top_locations <- top_locations %>%
  mutate(user_location = fct_reorder(user_location, user_count, .desc = TRUE)) %>%
  mutate(user_location = fct_rev(user_location))

ggplot(top_locations, aes(x = user_location, y = user_count)) +
  geom_col(fill = "steelblue") +
  geom_text(aes(label = user_count), hjust = -0.1, vjust = 0.5) +
  coord_flip() +
  labs(x = "User Location", y = "Number of Users") +
  theme_minimal() +
  scale_y_continuous(expand = expansion(mult = c(0, 0.15)))
```



1.2.4 Number of Tweets Associated with the Top 10 locations

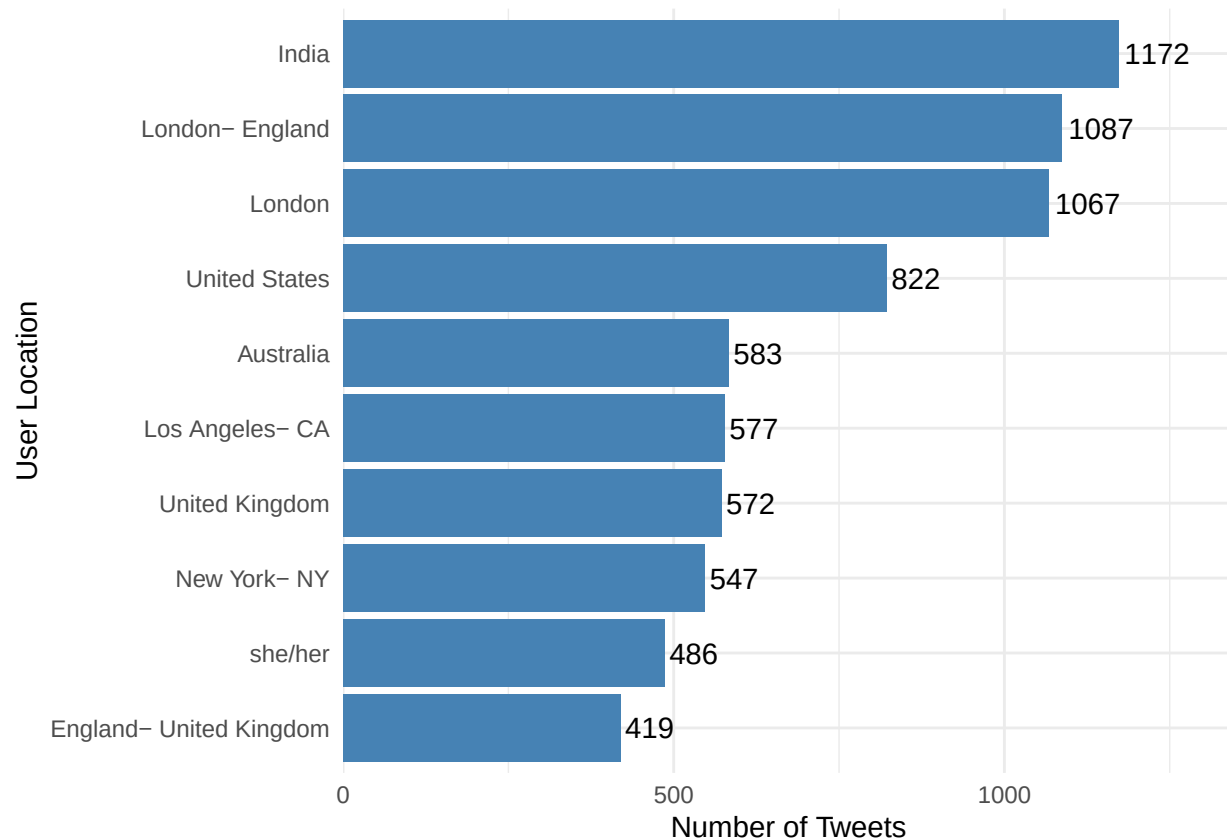
```

tweets_in_top_10_locations <- tweets %>%
  filter(user_location %in% top_locations$user_location) %>%
  group_by(user_location) %>%
  summarise(tweet_count = n()) %>%
  arrange(desc(tweet_count))

tweets_in_top_10_locations <- tweets_in_top_10_locations %>%
  mutate(user_location = fct_reorder(user_location, tweet_count, .desc = TRUE)) %>%
  mutate(user_location = fct_rev(user_location))

ggplot(tweets_in_top_10_locations, aes(x = user_location, y = tweet_count)) +
  geom_col(fill = "steelblue") +
  geom_text(aes(label = tweet_count), hjust = -0.1, vjust = 0.5) +
  coord_flip() +
  labs(x = "User Location", y = "Number of Tweets") +
  theme_minimal() +
  scale_y_continuous(expand = expansion(mult = c(0, 0.15)))

```



From the last two visualisations, the odd values are “she/her”, and some locations are the geographically the same from a broader perspective, but some are more specific to cities. For example, “New York- NY”, “Los Angeles- CA”, and “United States”. The first two locations are cities in the United States, but there is also “United States” as a location.

1.2.5 Bar Chart of the Length of Tweets

```

tweets <- tweets %>%
  mutate(tweet_length = nchar(text)) %>%
  mutate(length_group = case_when(
    tweet_length >= 1 & tweet_length <= 40 ~ "1-40",
    tweet_length >= 41 & tweet_length <= 80 ~ "41-80",
    tweet_length >= 81 & tweet_length <= 120 ~ "81-120",
    tweet_length >= 121 & tweet_length <= 160 ~ "121-160",
    tweet_length >= 161 & tweet_length <= 200 ~ "161-200",
    tweet_length >= 201 & tweet_length <= 240 ~ "201-240",
    tweet_length >= 241 ~ "241+",
    TRUE ~ "Unknown"
  ))

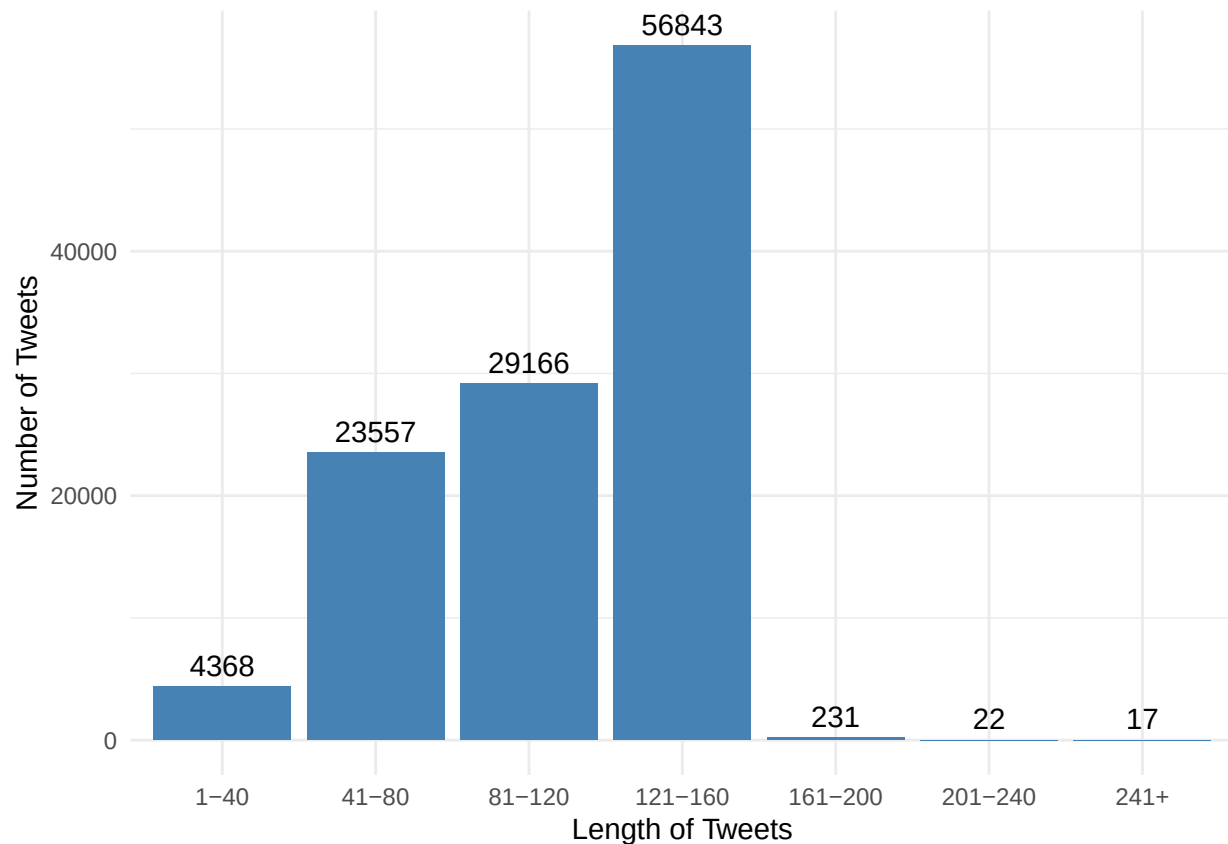
length_group_counts <- tweets %>%
  count(length_group)

custom_order <- c("1-40", "41-80", "81-120", "121-160", "161-200", "201-240", "241+")

```

```
length_group_counts <- length_group_counts %>%
  mutate(length_group = factor(length_group, levels = custom_order))

ggplot(length_group_counts, aes(x = length_group, y = n)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  geom_text(aes(label = n), vjust = -0.5) +
  labs(x = "Length of Tweets", y = "Number of Tweets") +
  theme_minimal()
```



1.2.6 Number of Tweets with Mentions and Its Count

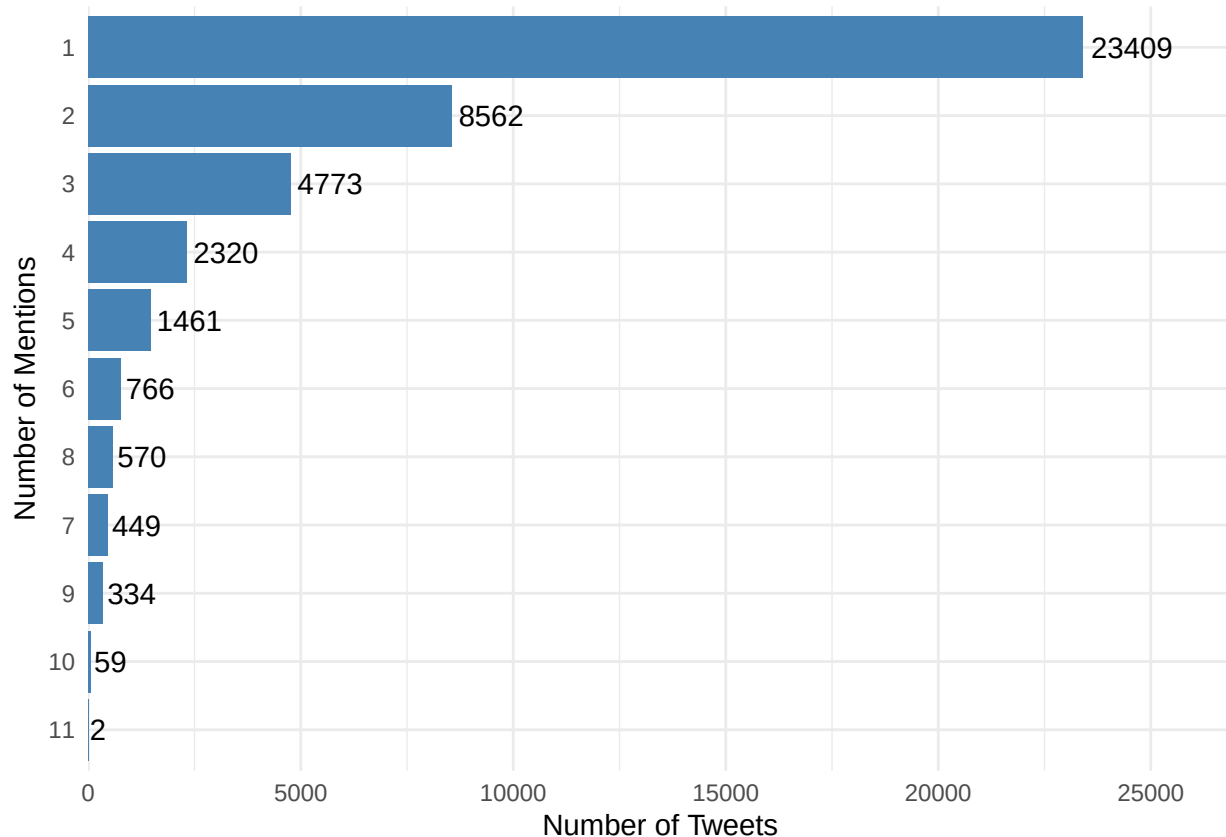
```
tweets_with_mentions <- tweets %>%
  filter(str_detect(text, "@[A-Za-z0-9_]+")) %>%
  mutate(mentions_count = str_count(text, "@[A-Za-z0-9_]+"))

tweets_with_mentions <- tweets_with_mentions %>%
  group_by(mentions_count) %>%
  summarise(tweet_count = n()) %>%
  arrange(desc(tweet_count))

tweets_with_mentions <- tweets_with_mentions %>%
  mutate(mentions_count = as.factor(mentions_count)) %>%
  mutate(mentions_count = fct_reorder(mentions_count, tweet_count, .desc = TRUE)) %>%
```

```
mutate(mentions_count = fct_rev(mentions_count))

ggplot(tweets_with_mentions, aes(x = mentions_count, y = tweet_count)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  geom_text(aes(label = tweet_count), hjust = -0.1, vjust = 0.5) +
  labs(x = "Number of Mentions", y = "Number of Tweets") +
  theme_minimal() +
  scale_y_continuous(expand = expansion(mult = c(0, 0.15)))
```



```
number_of_tweets_with_mentions <- tweets %>%
  filter(str_detect(text, "@[A-Za-z0-9_]+")) %>%
  nrow()

number_of_tweets_with_3_or_more_mentions <- tweets %>%
  filter(str_detect(text, "@[A-Za-z0-9_]+")) %>%
  mutate(mentions_count = str_count(text, "@[A-Za-z0-9_]+")) %>%
  filter(mentions_count >= 3) %>%
  nrow()

cat("Number of tweets with mentions: ", number_of_tweets_with_mentions, "\n")
```

```
## Number of tweets with mentions: 42705
```

```
cat("Number of tweets with 3 or more mentions: ", number_of_tweets_with_3_or_more_mentions, "\n")
```

```
## Number of tweets with 3 or more mentions: 10734
```

1.2.7 Top 20 Most Frequent Term in Tweets

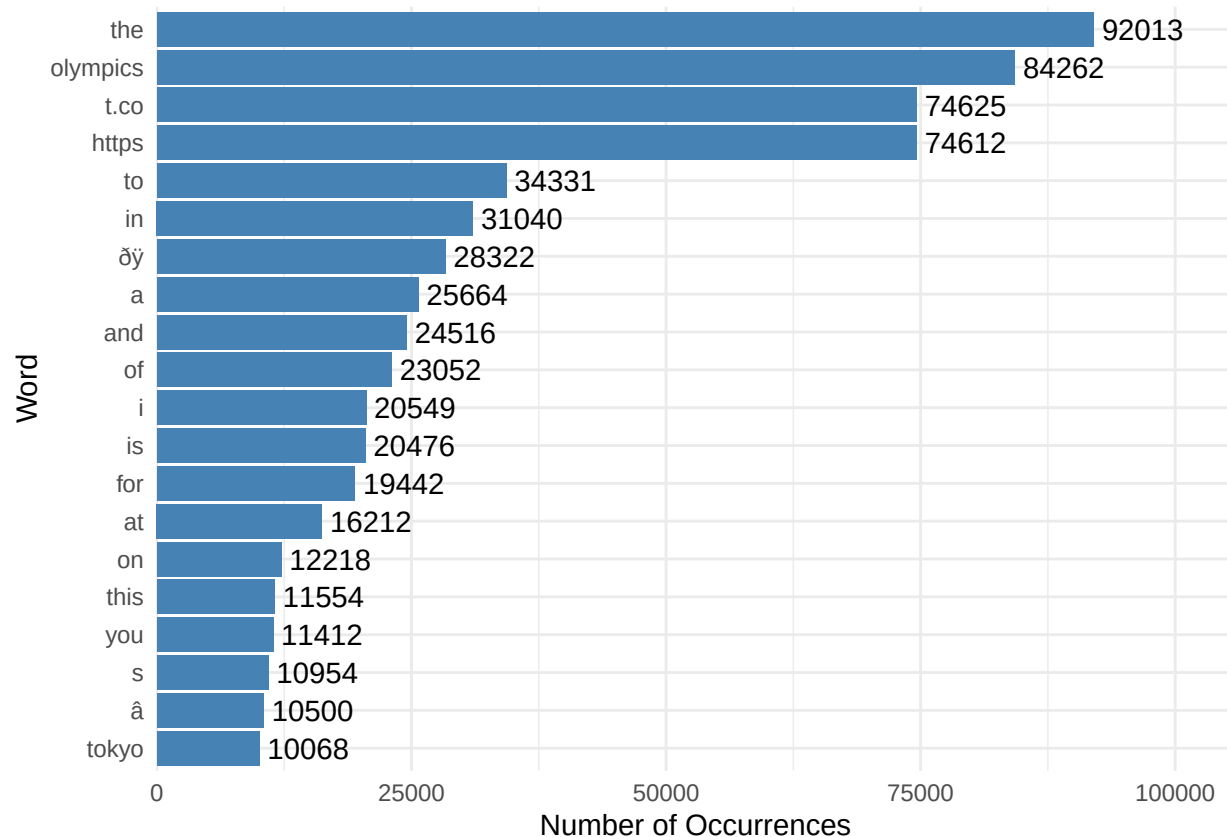
```
library(tidytext)

tweets_frequent_terms <- tweets %>%
  mutate(text = tolower(text)) %>%
  unnest_tokens(word, text) %>%
  count(word, sort = TRUE)

top_20_terms <- tweets_frequent_terms %>%
  dplyr::slice(1:20) %>%
  mutate(word = fct_reorder(word, n)) %>%
  mutate(word = fct_rev(word))

max_count <- max(top_20_terms$n)

ggplot(top_20_terms, aes(x = reorder(word, n), y = n)) +
  geom_col(fill = "steelblue") +
  geom_text(aes(label = n), hjust = -0.1, vjust = 0.5) +
  coord_flip() +
  labs(x = "Word", y = "Number of Occurrences") +
  theme_minimal() +
  scale_y_continuous(expand = expansion(mult = c(0, 0.15)))
```



1.2.8 Top 20 Most Frequent Non-Stopword Term in Tweets

```

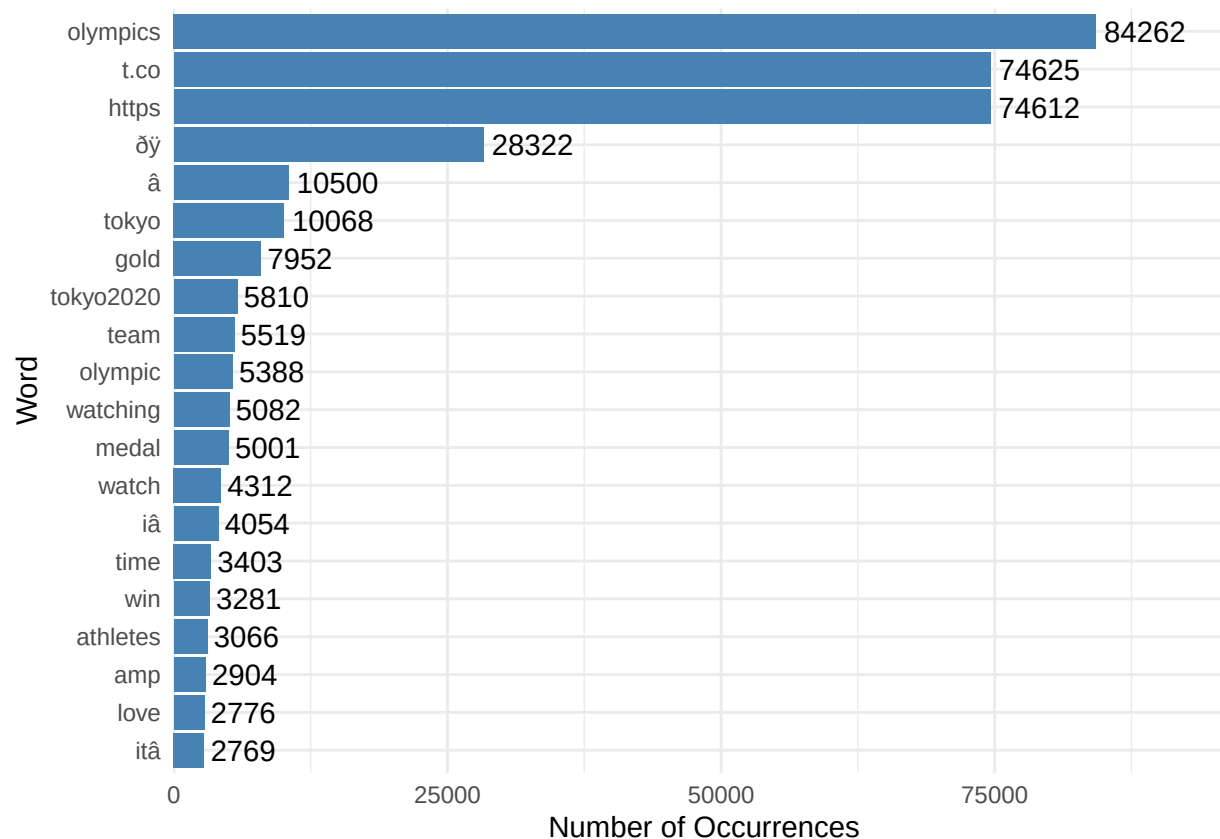
tweets_frequent_terms_no_stopwords <- tweets %>%
  mutate(text = tolower(text)) %>%
  unnest_tokens(word, text) %>%
  anti_join(stop_words) %>%
  count(word, sort = TRUE)

top_20_terms_no_stopwords <- tweets_frequent_terms_no_stopwords %>%
  dplyr::slice(1:20) %>%
  mutate(word = fct_reorder(word, n)) %>%
  mutate(word = fct_rev(word))

max_count_no_stopwords <- max(top_20_terms_no_stopwords$n)

ggplot(top_20_terms_no_stopwords, aes(x = reorder(word, n), y = n)) +
  geom_col(fill = "steelblue") +
  geom_text(aes(label = n), hjust = -0.1, vjust = 0.5) +
  coord_flip() +
  labs(x = "Word", y = "Number of Occurrences") +
  theme_minimal() +
  scale_y_continuous(expand = expansion(mult = c(0, 0.15)))

```



2 Predictive Data Analysis

2.1 Model Training

In this section, I will build machine learning models to predict whether a tweet is relevant or not. I will use the `glm` function to build a logistic regression models using all of the features from the dataset. The `glm` function is used to fit generalized linear models, which can be used for binary classification tasks like this one. Next, I will try to use interaction terms to see if that can improve the same model's performance. Finally, I will use the `xgboost` package to build a gradient boosting model to also evaluate how it performs compared to the logistic regression models.

First, I will make sure that the dependent variable is numeric and has only two values, 0 and 1. Luckily, the dataset has already been split into training and testing sets, so I will use the training set to build the models and the testing set to evaluate them.

```
# Load the training and testing datasets
train <- read.csv("predictive_twitter_train.csv")

# Check missing values in relevanceJudge column
```



```
cat("Missing values in relevanceJudge column: ",
    sum(is.na(train$relevanceJudge)), "\n")
```

```
## Missing values in relevanceJudge column: 0
```

```
# Check if relevanceJudge is numeric
cat("Is relevanceJudge numeric: ",
    is.numeric(train$relevanceJudge), "\n")
```

```
## Is relevanceJudge numeric: TRUE
```

```
# Number of columns in the training set
cat("Number of columns in the training set: ",
    ncol(train), "\n")
```

```
## Number of columns in the training set: 26
```

```
# Number of columns that are numeric in the training set
cat("Number of numeric columns in the training set: ",
    sum(sapply(train, is.numeric)), "\n")
```

```
## Number of numeric columns in the training set: 26
```

```
# Exclude tweetID from the training set
train <- train %>%
  select(-tweetID)

# Rename mistakenly named column nFavorties to nFavorites
train <- train %>%
  rename(nFavorites = nFavorties)
```

From the output in the code chunk above, we can see that the `relevanceJudge` column has no missing values, is numeric, and there are 26 columns in the training set, all of which are numeric. Next, I will build a logistic regression model using the `glm` function. I will use the `relevanceJudge` column as the dependent variable and all other columns as independent variables.

```
# Build the logistic regression model
modell1 <- glm(relevanceJudge ~ ., data = train, family = binomial)

# Summary of the model
summary(modell1)
```

```
##
## Call:
## glm(formula = relevanceJudge ~ ., family = binomial, data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -4.235e-01  1.332e-01  -3.178 0.001481 **
## text_score     1.778e-01  1.414e-02  12.569 < 2e-16 ***
```

```
## text_score_expansion 1.001e-01 1.340e-02 7.467 8.23e-14 ***
## hashtag 7.541e-02 5.716e-02 1.319 0.187098
## hasURL 1.195e+00 5.882e-02 20.321 < 2e-16 ***
## isReply -7.550e-01 1.237e-01 -6.105 1.03e-09 ***
## length 2.036e-04 7.731e-04 0.263 0.792248
## tweet_topic_time_diff -9.101e-03 5.197e-03 -1.751 0.079929 .
## semantic_overlap 9.292e-01 6.234e-02 14.905 < 2e-16 ***
## X.entityTypes 2.788e-01 6.843e-02 4.074 4.62e-05 ***
## X.entities 3.973e-02 1.778e-02 2.234 0.025489 *
## organization_entities -1.462e-01 6.233e-02 -2.345 0.019019 *
## person_entities -1.279e-01 6.132e-02 -2.087 0.036926 *
## work_entities -4.053e-01 7.101e-02 -5.708 1.14e-08 ***
## event_entities -2.652e+00 7.210e-01 -3.678 0.000235 ***
## species_entities -8.771e-01 3.137e-01 -2.796 0.005170 **
## places_entities -1.307e-01 6.412e-02 -2.039 0.041453 *
## nFollowers -2.921e-06 1.337e-06 -2.185 0.028874 *
## nFriends 6.504e-06 3.846e-06 1.691 0.090849 .
## nFavorites 2.204e-06 6.977e-06 0.316 0.752098
## nListed 9.748e-05 4.444e-05 2.193 0.028285 *
## isVerified 7.840e-01 2.594e-01 3.023 0.002506 **
## isGeoEnabled 1.067e-02 5.493e-02 0.194 0.845962
## twitterAge 1.672e-01 2.599e-02 6.434 1.24e-10 ***
## X.tweetsPosted -1.664e-06 4.380e-07 -3.799 0.000146 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 18338 on 35846 degrees of freedom
## Residual deviance: 14354 on 35822 degrees of freedom
## (112 observations deleted due to missingness)
## AIC: 14404
##
## Number of Fisher Scoring iterations: 7
```

```
# Check the coefficients of the model
cat("Coefficients of the model with all of the features: \n")
```

```
## Coefficients of the model with all of the features:
```

```
print(coef(model1))
```

```
##          (Intercept)          text_score text_score_expansion
##      -4.235116e-01      1.777828e-01      1.000739e-01
##          hashtag          hasURL          isReply
##      7.541028e-02      1.195298e+00      -7.549720e-01
##          length tweet_topic_time_diff      semantic_overlap
##      2.036337e-04      -9.101056e-03      9.292260e-01
##      X.entityTypes      X.entities organization_entities
##      2.787911e-01      3.973013e-02      -1.461808e-01
##      person_entities      work_entities      event_entities
##      -1.279425e-01      -4.053250e-01      -2.651862e+00
##      species_entities      places_entities      nFollowers
```

```
##          -8.771407e-01          -1.307291e-01          -2.921211e-06
##              nFriends              nFavorites              nListed
##          6.503686e-06          2.203739e-06          9.747565e-05
##          isVerified          isGeoEnabled          twitterAge
##          7.839781e-01          1.067119e-02          1.672066e-01
##      X.tweetsPosted
##          -1.663699e-06
```

Next, I will build another model with interaction terms between some of the author's account details.

```
# Build the logistic regression model with interaction terms
model2 <- glm(relevanceJudge ~ isVerified *
              nFollowers *
              nFriends *
              nFavorites *
              nListed +
              .,
              data = train, family = binomial)

# Summary of the model
summary(model2)
```

```
##
## Call:
## glm(formula = relevanceJudge ~ isVerified * nFollowers * nFriends *
##      nFavorites * nListed + ., family = binomial, data = train)
##
## Coefficients:
##
##              Estimate Std. Error
## (Intercept)    -2.001e+15  2.324e+06
## isVerified      3.659e+14  6.647e+06
## nFollowers     -3.262e+09  3.691e+01
## nFriends        8.710e+08  1.722e+02
## nFavorites      8.264e+09  3.544e+02
## nListed         1.140e+11  1.847e+03
## text_score      1.266e+13  2.605e+05
## text_score_expansion -4.175e+13  2.446e+05
## hashtag         6.350e+13  9.147e+05
## hasURL          3.905e+14  7.981e+05
## isReply        -5.003e+14  1.135e+06
## length          9.043e+11  1.113e+04
## tweet_topic_time_diff -5.086e+12  8.387e+04
## semantic_overlap   7.829e+14  1.600e+06
## X.entityTypes     1.392e+14  1.113e+06
## X.entities        2.752e+13  2.889e+05
## organization_entities -7.449e+13  1.072e+06
## person_entities   -7.642e+13  1.038e+06
## work_entities     -2.155e+14  9.700e+05
## event_entities    -2.775e+15  5.325e+06
## species_entities   -3.476e+14  2.993e+06
## places_entities   -1.048e+14  1.101e+06
## isGeoEnabled      2.609e+13  8.423e+05
## twitterAge        7.114e+13  4.193e+05
```

## X.tweetsPosted	-5.089e+08	6.484e+00
## isVerified:nFollowers	3.437e+09	3.997e+01
## isVerified:nFriends	1.352e+10	1.683e+03
## nFollowers:nFriends	-4.900e+04	2.446e-03
## isVerified:nFavorites	-4.304e+11	1.265e+04
## nFollowers:nFavorites	1.423e+06	4.601e-02
## nFriends:nFavorites	-4.262e+06	1.066e-01
## isVerified:nListed	-1.132e+11	2.052e+03
## nFollowers:nListed	-6.598e+04	1.308e-03
## nFriends:nListed	-4.837e+05	1.350e-01
## nFavorites:nListed	-1.420e+07	4.496e-01
## isVerified:nFollowers:nFriends	-4.276e+05	5.439e-03
## isVerified:nFollowers:nFavorites	-1.025e+07	2.281e-01
## isVerified:nFriends:nFavorites	7.432e+07	9.445e+00
## nFollowers:nFriends:nFavorites	5.028e+01	1.341e-06
## isVerified:nFollowers:nListed	5.328e+04	1.336e-03
## isVerified:nFriends:nListed	1.384e+07	2.905e-01
## nFollowers:nFriends:nListed	3.158e+01	8.185e-07
## isVerified:nFavorites:nListed	2.114e+08	8.324e+00
## nFollowers:nFavorites:nListed	-3.186e+02	1.005e-05
## nFriends:nFavorites:nListed	3.824e+03	1.180e-04
## isVerified:nFollowers:nFriends:nFavorites	-1.693e+02	6.038e-05
## isVerified:nFollowers:nFriends:nListed	-2.219e+01	8.612e-07
## isVerified:nFollowers:nFavorites:nListed	6.118e+02	1.346e-05
## isVerified:nFriends:nFavorites:nListed	-2.775e+04	9.404e-04
## nFollowers:nFriends:nFavorites:nListed	-5.248e-02	1.204e-09
## isVerified:nFollowers:nFriends:nFavorites:nListed	9.821e-02	3.559e-09
##	z value	Pr(> z)
## (Intercept)	-861014825	<2e-16 ***
## isVerified	55041097	<2e-16 ***
## nFollowers	-88376849	<2e-16 ***
## nFriends	5057530	<2e-16 ***
## nFavorites	23320687	<2e-16 ***
## nListed	61755358	<2e-16 ***
## text_score	48596580	<2e-16 ***
## text_score_expansion	-170697179	<2e-16 ***
## hashtag	69426817	<2e-16 ***
## hasURL	489279939	<2e-16 ***
## isReply	-440820538	<2e-16 ***
## length	81213733	<2e-16 ***
## tweet_topic_time_diff	-60642376	<2e-16 ***
## semantic_overlap	489223003	<2e-16 ***
## X.entityTypes	125121068	<2e-16 ***
## X.entities	95239214	<2e-16 ***
## organization_entities	-69465694	<2e-16 ***
## person_entities	-73589546	<2e-16 ***
## work_entities	-222127214	<2e-16 ***
## event_entities	-521171025	<2e-16 ***
## species_entities	-116130170	<2e-16 ***
## places_entities	-95130244	<2e-16 ***
## isGeoEnabled	30970959	<2e-16 ***
## twitterAge	169654654	<2e-16 ***
## X.tweetsPosted	-78479908	<2e-16 ***
## isVerified:nFollowers	85987831	<2e-16 ***

```
## isVerified:nFriends      8033818 <2e-16 ***
## nFollowers:nFriends     -20031292 <2e-16 ***
## isVerified:nFavorites   -34018918 <2e-16 ***
## nFollowers:nFavorites    30939253 <2e-16 ***
## nFriends:nFavorites     -39987431 <2e-16 ***
## isVerified:nListed      -55178701 <2e-16 ***
## nFollowers:nListed      -50458594 <2e-16 ***
## nFriends:nListed        -3583594 <2e-16 ***
## nFavorites:nListed      -31580848 <2e-16 ***
## isVerified:nFollowers:nFriends -78619672 <2e-16 ***
## isVerified:nFollowers:nFavorites -44929033 <2e-16 ***
## isVerified:nFriends:nFavorites  7869117 <2e-16 ***
## nFollowers:nFriends:nFavorites  37503540 <2e-16 ***
## isVerified:nFollowers:nListed  39891051 <2e-16 ***
## isVerified:nFriends:nListed  47647445 <2e-16 ***
## nFollowers:nFriends:nListed  38576780 <2e-16 ***
## isVerified:nFavorites:nListed  25396917 <2e-16 ***
## nFollowers:nFavorites:nListed -31708131 <2e-16 ***
## nFriends:nFavorites:nListed  32414860 <2e-16 ***
## isVerified:nFollowers:nFriends:nFavorites -2804761 <2e-16 ***
## isVerified:nFollowers:nFriends:nListed -25762649 <2e-16 ***
## isVerified:nFollowers:nFavorites:nListed  45445230 <2e-16 ***
## isVerified:nFriends:nFavorites:nListed -29505961 <2e-16 ***
## nFollowers:nFriends:nFavorites:nListed -43586628 <2e-16 ***
## isVerified:nFollowers:nFriends:nFavorites:nListed  27591854 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 18338 on 35846 degrees of freedom
## Residual deviance: 187283 on 35796 degrees of freedom
## (112 observations deleted due to missingness)
## AIC: 187385
##
## Number of Fisher Scoring iterations: 25
```

```
# Check the coefficients of the model
```

```
cat("Coefficients of the model with interaction terms:\n")
```

```
## Coefficients of the model with interaction terms:
```

```
print(coef(model12))
```

```
## (Intercept)
## -2.001074e+15
## isVerified
## 3.658789e+14
## nFollowers
## -3.261598e+09
## nFriends
## 8.710216e+08
## nFavorites
```

```

##                8.264190e+09
##                nListed
##                1.140477e+11
##                text_score
##                1.265763e+13
##                text_score_expansion
##                -4.174894e+13
##                hashtag
##                6.350248e+13
##                hasURL
##                3.905069e+14
##                isReply
##                -5.003463e+14
##                length
##                9.042940e+11
##                tweet_topic_time_diff
##                -5.086181e+12
##                semantic_overlap
##                7.829045e+14
##                X.entityTypes
##                1.392449e+14
##                X.entities
##                2.751816e+13
##                organization_entities
##                -7.448659e+13
##                person_entities
##                -7.642109e+13
##                work_entities
##                -2.154590e+14
##                event_entities
##                -2.775456e+15
##                species_entities
##                -3.475793e+14
##                places_entities
##                -1.047507e+14
##                isGeoEnabled
##                2.608690e+13
##                twitterAge
##                7.114189e+13
##                X.tweetsPosted
##                -5.088577e+08
##                isVerified:nFollowers
##                3.436551e+09
##                isVerified:nFriends
##                1.351915e+10
##                nFollowers:nFriends
##                -4.900018e+04
##                isVerified:nFavorites
##                -4.303692e+11
##                nFollowers:nFavorites
##                1.423368e+06
##                nFriends:nFavorites
##                -4.262042e+06
##                isVerified:nListed

```

```
## -1.132383e+11
## nFollowers:nListed
## -6.598140e+04
## nFriends:nListed
## -4.837147e+05
## nFavorites:nListed
## -1.420013e+07
## isVerified:nFollowers:nFriends
## -4.276496e+05
## isVerified:nFollowers:nFavorites
## -1.024971e+07
## isVerified:nFriends:nFavorites
## 7.432456e+07
## nFollowers:nFriends:nFavorites
## 5.027845e+01
## isVerified:nFollowers:nListed
## 5.327658e+04
## isVerified:nFriends:nListed
## 1.384104e+07
## nFollowers:nFriends:nListed
## 3.157517e+01
## isVerified:nFavorites:nListed
## 2.114039e+08
## nFollowers:nFavorites:nListed
## -3.186386e+02
## nFriends:nFavorites:nListed
## 3.823506e+03
## isVerified:nFollowers:nFriends:nFavorites
## -1.693415e+02
## isVerified:nFollowers:nFriends:nListed
## -2.218705e+01
## isVerified:nFollowers:nFavorites:nListed
## 6.117705e+02
## isVerified:nFriends:nFavorites:nListed
## -2.774661e+04
## nFollowers:nFriends:nFavorites:nListed
## -5.247893e-02
## isVerified:nFollowers:nFriends:nFavorites:nListed
## 9.820808e-02
```

Finally, I will build an XGboost model using the xgboost package.

```
# Prepare the data for XGBoost
train_clean <- na.omit(train) # Remove rows with NA values
train_matrix <- model.matrix(relevanceJudge ~ ., data = train_clean)[, -1]
train_label <- train_clean$relevanceJudge

# Build the XGBoost model
model3 <- xgboost(data = train_matrix, label = train_label,
                  nrounds = 100, objective = "binary:logistic",
                  eval_metric = "logloss", verbose = 0)

# Check the importance of the features
cat("Feature importance of the XGBoost model: \n")
```

Feature importance of the XGBoost model:

```
print(xgb.importance(model = model3))
```

##	Feature	Gain	Cover	Frequency
##	<char>	<num>	<num>	<num>
## 1:	text_score	0.2971570070	0.1555051801	0.119691120
## 2:	text_score_expansion	0.1802374250	0.1417552313	0.107004964
## 3:	tweet_topic_time_diff	0.0625458694	0.0605354235	0.072807501
## 4:	hasURL	0.0621510770	0.0577907258	0.011307226
## 5:	X.tweetsPosted	0.0597681346	0.0886758516	0.113623828
## 6:	twitterAge	0.0574150501	0.0863005188	0.104522890
## 7:	length	0.0470828167	0.0736330540	0.078323221
## 8:	nFollowers	0.0456286280	0.0763901833	0.079150579
## 9:	nFriends	0.0414729255	0.0539635232	0.080253723
## 10:	nListed	0.0310346252	0.0497134824	0.058190844
## 11:	nFavorites	0.0227244700	0.0245656689	0.046607832
## 12:	semantic_overlap	0.0159096933	0.0251220945	0.013789300
## 13:	person_entities	0.0127321428	0.0143905480	0.014340871
## 14:	X.entities	0.0119412307	0.0140357012	0.024544953
## 15:	X.entityTypes	0.0110749448	0.0026996609	0.018201875
## 16:	organization_entities	0.0094986235	0.0104875430	0.010755654
## 17:	places_entities	0.0090772261	0.0095262741	0.012686156
## 18:	work_entities	0.0065161663	0.0091651967	0.009376724
## 19:	isReply	0.0059166654	0.0129343865	0.004412576
## 20:	hashtag	0.0044604834	0.0039370016	0.007722008
## 21:	isGeoEnabled	0.0032352519	0.0009733409	0.006343078
## 22:	event_entities	0.0012040671	0.0151891331	0.003033646
## 23:	isVerified	0.0007598629	0.0033126639	0.001378930
## 24:	species_entities	0.0004556132	0.0093976125	0.001930502
##	Feature	Gain	Cover	Frequency

2.2 Model Evalutaion

Now I will evaluate the performance of the models by measuring their F1 score.

```
# Make predictions on the train set using the models just to evaluate their performance.  
# I will use the train set later for the final model's predictions.  
predictions_model1 <- predict(model1, newdata = train, type = "response")  
predictions_model2 <- predict(model2, newdata = train, type = "response")  
predictions_model3 <- predict(model3, newdata = train_matrix)  
  
# Convert predictions to binary values (0 or 1) using a threshold of 0.5  
predictions_model1_binary <- ifelse(predictions_model1 > 0.5, 1, 0)  
predictions_model2_binary <- ifelse(predictions_model2 > 0.5, 1, 0)  
predictions_model3_binary <- ifelse(predictions_model3 > 0.5, 1, 0)  
  
# Get the F1 score for each model  
f1_score_model1 <- F1_Score(y_true = train$relevanceJudge, y_pred = predictions_model1_binary)  
f1_score_model2 <- F1_Score(y_true = train$relevanceJudge, y_pred = predictions_model2_binary)  
f1_score_model3 <- F1_Score(y_true = train_label, y_pred = predictions_model3_binary)
```



```
# Print the F1 scores
cat("F1 Score for Model 1 (Logistic Regression): ", f1_score_model1, "\n")
cat("F1 Score for Model 2 (Logistic Regression with interaction terms): ", f1_score_model2, "\n")
cat("F1 Score for Model 3 (XGBoost Model): ", f1_score_model3, "\n")
```

```
## F1 Score for Model 1 (Logistic Regression): 0.9638022
```

```
## F1 Score for Model 2 (Logistic Regression with interaction terms): 0.9623675
```

```
## F1 Score for Model 3 (XGBoost Model): 0.9854909
```

From the output of the code chunk above, we can see that the first and the second model that used logistic regression have the same F1 score rounded up to 2 decimal places - 0.96. The third model that used XGBoost performed better than the preceding two models with an F1 score of 0.99, rounded up to 2 decimal places.

2.3 Final Model Improvements

I will now try to improve the XGBoost model further, which has an exact F1 score of 0.9854909. I will filter out any outliers in the dataset, deal with missing values, and select the most important features based on the feature importance from the XGBoost model.

```
# Number of rows before filtering outliers and handling missing values
cat("Number of rows before filtering outliers: ", nrow(train), "\n")
```

```
## Number of rows before filtering outliers: 35959
```

```
# Columns to omit rows with NA values
columns_for_na_omit <- c("length", "nFollowers", "nFriends", "isVerified")
```

```
# Columns to calculate mean for missing values
columns_for_mean <- c("text_score", "text_score_expansion",
  "twitterAge", "X.tweetsPosted")
```

```
# Columns to get mode for missing values
columns_for_mode <- c("nFavorites", "nListed")
```

```
# Handle missing values by getting either omitting the row, calculating the mean, or getting the mode o.
train_filtered <- train %>%
  filter(if_all(all_of(columns_for_na_omit), ~ !is.na(.)))
```

```
train_filtered <- train_filtered %>%
  mutate(across(all_of(columns_for_mean),
    ~ ifelse(is.na(.), mean(., na.rm = TRUE), .)))
```

```
train_filtered <- train_filtered %>%
  mutate(across(all_of(columns_for_mode),
    ~ ifelse(is.na(.), as.numeric(names(sort(table(.), decreasing = TRUE)[1])), .)))
```

```
# Number of rows after handling missing values
cat("Number of rows after handling missing values: ", nrow(train_filtered), "\n")
```

```
## Number of rows after handling missing values: 35959
```

```
# Select columns to filter outliers
columns_to_filter <- c("text_score", "text_score_expansion",
  "length", "nFollowers", "nFriends",
  "nFavorites", "nListed", "twitterAge", "X.tweetsPosted")

# Filter out outliers based on the interquartile range (IQR)
train_filtered <- train_filtered %>%
  filter(across(all_of(columns_to_filter), ~ . >= quantile(., 0.05) & . <= quantile(., 0.95)))

# Number of rows after filtering outliers
cat("Number of rows after filtering outliers: ", nrow(train_filtered), "\n")
```

```
## Number of rows after filtering outliers: 22300
```

Now I will build the final XGBoost model using the filtered dataset and the most important features from the previous XGBoost model.

```
# Prepare the data for XGBoost
importance_matrix <- xgb.importance(model = model3)

# Select the 10 most important features
top_features <- importance_matrix$Feature[1:10]
train_matrix_final <- model.matrix(relevanceJudge ~ ., data = train_filtered[, c(top_features, "relevanceJudge")])
train_label_final <- train_filtered$relevanceJudge

# Build the final XGBoost model
model_final <- xgboost(data = train_matrix_final, label = train_label_final,
  nrounds = 100, objective = "binary:logistic",
  eval_metric = "logloss", verbose = 0)

# Check the importance of the features in the final model
cat("Feature importance of the final XGBoost model: \n")
```

```
## Feature importance of the final XGBoost model:
```

```
print(xgb.importance(model = model_final))
```

```
##           Feature      Gain    Cover  Frequency
##           <char>      <num>    <num>    <num>
## 1:      text_score 0.24747403 0.14198831 0.10829639
## 2: text_score_expansion 0.18903638 0.15134988 0.12539582
## 3:   X.tweetsPosted 0.10234047 0.12643780 0.15167828
## 4:   twitterAge 0.09287731 0.16040045 0.14407853
## 5:         nFriends 0.07167041 0.07468873 0.11082964
## 6:     nFollowers 0.07070947 0.09603792 0.11304623
## 7:         length 0.06666916 0.06894439 0.10006333
## 8: tweet_topic_time_diff 0.05995971 0.04599729 0.06269791
## 9:         hasURL 0.05434089 0.07124094 0.01519949
## 10:      nListed 0.04492218 0.06291430 0.06871438
```

```
# Check the F1 score of the final model
predictions_final <- predict(model_final, newdata = train_matrix_final)
predictions_final_binary <- ifelse(predictions_final > 0.5, 1, 0)
f1_score_final <- F1_Score(y_true = train_label_final, y_pred = predictions_final_binary)
cat("F1 Score for the final XGBoost model: ", f1_score_final, "\n")
```

```
## F1 Score for the final XGBoost model: 0.9880673
```

By handling missing values and filtering out outliers, the final XGBoost model has an exact F1 score of 0.9880673, which is a 0.0025764 improvement from the previous XGBoost model's F1 score of 0.9854909.

2.3.1 Justifications of the Final Model

As per (Clark & Lee, n.d.), “Gradient boosting is an ensemble learning algorithm that produces accurate predictions by combining multiple decision trees into a single model.” Essentially, the algorithm builds a series of decision trees that corrects the errors of the preceding trees, which makes it a powerful model for complex datasets. I figured that this type of algorithm is particularly useful for this dataset because a decision tree applies for such classification tasks.

Hence, I've decided to use XGBoost model as one of the three models that I built because it is a powerful implementation of gradient boosting that can handle datasets of all sizes and complex relationships between features. It also has built-in regularization to prevent overfitting, which is important for this dataset with many features. Given that this is a provided dataset and I do not have any prior knowledge of the data, XGBoost was a great choice as it does well in handling datasets where the relationship between predictors and the outcome is complex. (Hachcham (n.d.))

As mentioned earlier, the initial XGBoost model already had an F1 score of 0.99, rounded up to 2 decimal places. There wasn't much room for improvement, but I was able to improve the F1 score by 0.0025764 by filtering out outliers and handling missing values. I chose follow that method because it is a common yet a reliably effective way to improve the performance of a model, even if it already has an accuracy of close to 100%. And since I haven't performed those preprocessing steps yet, I thought it was the best choice to improve the model's performance.

2.4 Predictions on the Testing Set

Finally, I will use the final XGBoost model to make predictions on the testing set and save the results in its corresponding CSV file.

```
library(dplyr)

original_test <- read.csv("predictive_twitter_test.csv")
test <- original_test %>%
  rename(nFavorites = nFavorties)

for (col in top_features) {
  if (is.numeric(test[[col]])) {
    test[[col]][is.na(test[[col]])] <- mean(test[[col]], na.rm = TRUE)
  }
}

# Make model matrix for XGBoost and predict
test_matrix <- model.matrix(~ . - 1, data = test[, top_features])
```

```
predictions <- predict(model_final, newdata = test_matrix)
predictions_binary <- ifelse(predictions > 0.5, 1, 0)

original_test$relevanceJudge <- predictions_binary
write.csv(original_test, "predictive_twitter_test.csv", row.names = FALSE)

cat("Number of relevant tweets in the test set: ",
    sum(original_test$relevanceJudge == 1), "\n")
```

```
## Number of relevant tweets in the test set: 250
```

```
cat("Number of irrelevant tweets in the test set: ",
    sum(original_test$relevanceJudge == 0), "\n")
```

```
## Number of irrelevant tweets in the test set: 3746
```

References

- Clark, B., & Lee, F. (n.d.). *What is Gradient Boosting?* <https://www.ibm.com/think/topics/gradient-boosting>
- Hachcham, A. (n.d.). *XGBoost: Everything You Need to Know.* <https://neptune.ai/blog/xgboost-everything-you-need-to-know>